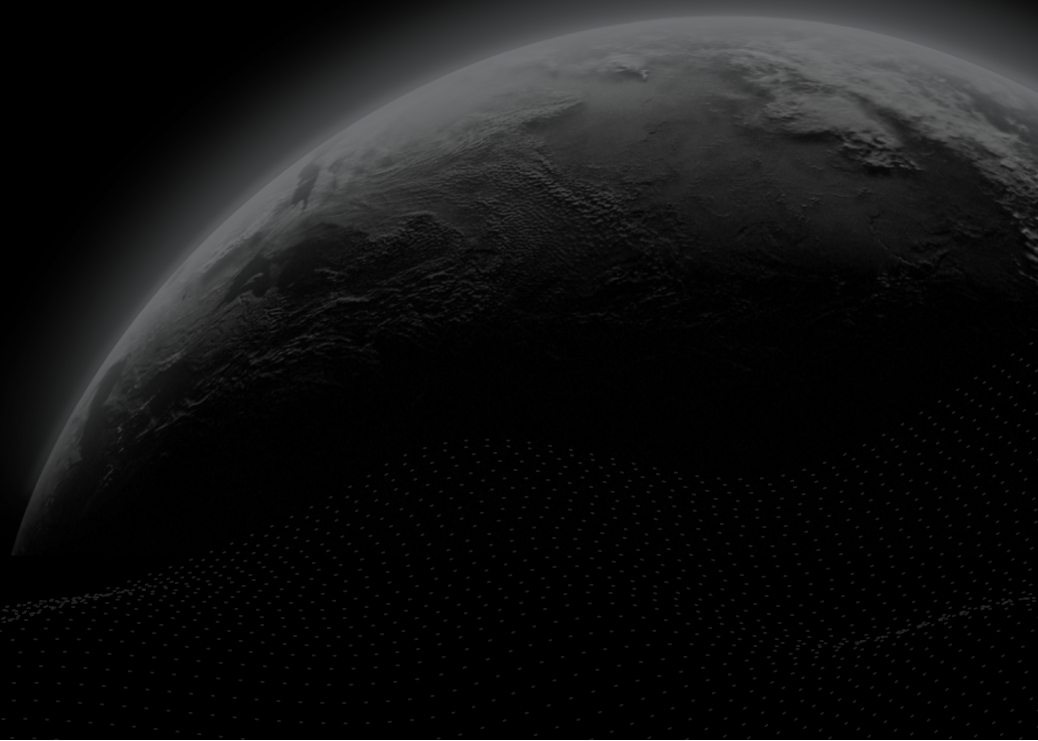
CERTIK

Security Assessment

NATIX Network - audit

CertiK Assessed on Sept 19th, 2024



CertiK Assessed on Sept 19th, 2024

NATIX Network - audit

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Staking	Solana (SOL)	Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Rust	Delivered on 09/19/2024	N/A

CODEBASE

saphira

View All in Codebase Page

COMMITS

679866c484b6824872df0f9bba3301611a0a65ab

3db9356d300f1850fc31ba0df844b1aecb094225

dd18be8dcb430031e40de0a26a1de28f0ab9ddd6

View All in Codebase Page

Vulnerability Summary



14	9	0	0	5	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
1	Major	1 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
3	Medium	3 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
3	Minor	3 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
7	Informational	3 Resolved, 4 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | NATIX NETWORK - AUDIT

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

System Overview

External Dependencies

Privileged Functions

Findings

GLOBAL-01 : Centralization Related Risks and Upgradability

COS-01 : Weak Validation on Token Account Owner

CVB-01 : Compilation Failure

LIB-03 : Potential Missing Link Between `program_account` and `stake_account`

CHA-01 : Unbounded range for the interest rate

SRC-03 : Missing Owner Validation on `user_account` and `stake_account`

WIT-01 : Potential incorrect `forfeit_amount` implementation

CAN-01 : Potential Missing Reward Handling While Gap between `unstake` and `cancel`

GER-01 : Unknown source of the reward tokens

GER-02 : Concerns on the reward calculation

GIV-01 : Concerns on compute unit and Account Size

LIB-01 : Missing Address Check for Token Program

LIB-02 : Missing validation on mint account for `stake_token_account`

SRC-02 : Cancellation may be blocked due to stake amount limit validation And Break `maximum_stakers` Limitation

Optimizations

SRC-01 : Potential Conflict on Setting

Appendix

Disclaimer

CODEBASE | NATIX NETWORK - AUDIT

Repository

saphira

Commit

679866c484b6824872df0f9bba3301611a0a65ab

3db9356d300f1850fc31ba0df844b1aecb094225

dd18be8dcb430031e40de0a26a1de28f0ab9ddd6

9bf1e6e4f3b190b28e077baba901858c7aa9565b

309c59f2a64cdb7d165cb6d1ca11db55a293df5a

2bf22d4622ecf96a88cf3950d59e0cc706f5312f

AUDIT SCOPE | NATIX NETWORK - AUDIT

32 files audited ● 5 files with Acknowledged findings ● 6 files with Resolved findings ● 21 files without findings

ID	Repo	File	SHA256 Checksum
BUR	natixnetwork/saphira	src/burn_or_transfer.rs	b8b0f22b090a3fd58b5040ee28051a5b8ae19ec8de69e1a536ddf3735fbb3b1c
CAN	natixnetwork/saphira	src/cancel.rs	bcd751228cd1c7c94604d299c57b02b36bf0b7e3cbb17b7ffe0c4a2d626f5b3d
GER	natixnetwork/saphira	src/get_reward.rs	0a4b001c8a281de3094ca88131653d90ebec6e4f0b18a4210c85ec3eca018cd9
GIV	natixnetwork/saphira	src/give_stakes.rs	8be8d873ca0e7f93930614c9a73d81008da46dec848d2ed818d7368ff7a244ee
SET	natixnetwork/saphira	src/set_config.rs	f9f08a18867fb79875d5130abdf6458ef417f91d4d1ba9b45f9f5f99a5e45db2
CHA	natixnetwork/saphira	src/change_interest_rate.rs	266da438189713771160173a3ad5a32a5907cf4d94aab166c92143bd5b809eda
COL	natixnetwork/saphira	src/control_stake_amount.rs	2d7970914d2a7cc59975da217fd207227eff9d7887a4d1a83a7082e75186dc84
COU	natixnetwork/saphira	src/control_user_account.rs	8d963bfff8700ae9ffadde685b082dd29abbfbe5d65c3fe1784015f0d0ce2dd3
COS	natixnetwork/saphira	src/control_user_solana_address.rs	170a7f771fd211afe416fbd329dc4d06064e3e6b1924f138b4e0a809dc11ad69
LIB	natixnetwork/saphira	src/lib.rs	8916051152ba9f2d4c55d243cf1e733f70208707d7039cced2ec51066f609313
WIT	natixnetwork/saphira	src/withdraw.rs	62e7566701faaf850c5d40899f5a1c224c16c22eb6833e9c32032a660cc778e8
CON	natixnetwork/saphira	src/control_flow.rs	74a80dbc30392f26966a5d7a6a819d8b2e5a95cee83495a987c185d5cfc4ef51
COT	natixnetwork/saphira	src/control_max_stakers.rs	35d78e58b8db83a1730255b7986a1dd5fc9e46657c957cd1c95dd3fdf027786b
COR	natixnetwork/saphira	src/control_owner.rs	95a3e3dfafe67b8d8bab7a2c1c72037e17e6d7315344148a9e1c6a5080186053

ID	Repo	File	SHA256 Checksum
COO	natixnetwork/saphira	src/control_provisioned.rs	0830e1e19552a19f48dacc855c3deaacb649a0be0646213e37df75856f8b6fda
DAT	natixnetwork/saphira	src/data.rs	a4b5ad62d81fd4356bb92862239cf1646cec6f7ef9bd0932a2abe18c8635b7e4
DES	natixnetwork/saphira	src/deserialize.rs	820145c8fe449be87cbe0d2bb4cdaf680fa6b49b16011d8ffa87f6a96b18a85a
GET	natixnetwork/saphira	src/get_interest_rate.rs	b182024abaa52098099b9ddda28769e2b00a8f2eac15a403587f3520ea7de52d
GEN	natixnetwork/saphira	src/get_natix_token_id.rs	0803485564519fd564a233277f3482546361c7bba0f85bff57b34dbee17b549c
GEA	natixnetwork/saphira	src/get_natix_token_mint.rs	8e9b2f07b19054539755929f35dd157bc92530f69b46ef046ac08106ef4de403
GEO	natixnetwork/saphira	src/get_owner_id.rs	390e79f7de238a1f6341dae3291171bbedebd64d2e7bea5ec967d2fbf2470316
GEP	natixnetwork/saphira	src/get_pda.rs	20b549442f677d7ee48b0c559ca081fdc2c39c8b3c5984a172778d36118b8a0d
GIE	natixnetwork/saphira	src/give_unstakes.rs	08bb7666dcc8c8cf14207333f72756e80b5699d9b19370566702869d22c314c0
INI	natixnetwork/saphira	src/init.rs	c58e10fbba5e664872fe18e1edd2c08810e82f5146bb36da0eb6acbe9c8a17ba
INS	natixnetwork/saphira	src/instruction.rs	9eef3ab4e5db4f844fb84a2e85d5fb1f211181beafb7bda2b458d3e718585f21
PAU	natixnetwork/saphira	src/pause.rs	09f6d050a1ec72792067250c3a44ecd861e5e8a26632c30fca35249805178 4f9
RES	natixnetwork/saphira	src/resume.rs	150dcf2041d7b4ca1870f169e077db8f66fc70d2479060c26330ea62c4633c42
SER	natixnetwork/saphira	src/serialize.rs	8fdfb46443e335b24b0394f4754f981a973ed805176ca7a403e71d28e6382c5e
STA	natixnetwork/saphira	src/stake.rs	bf6f52b59912129036ce5c2097d109deb638105f57f07d7905066aba6f921509
STK	natixnetwork/saphira	src/staking_info.rs	0633ada2345434350c2a4ac9b6ea614f0f1350d5fceb99160d39b23904eab24c
TRA	natixnetwork/saphira	src/transfer_token.rs	ef0bf6b0659a14ae139b8d49d22c287b82a1a0cfd6f2955986ff02b2f1de5794

ID	Repo	File	SHA256 Checksum
UNS	natixnetwork/saphira	src/unstake.rs	37b5404f8db665b6f0ee45d2bb5332efc95e4480093403a2914503bcff9d36ad

APPROACH & METHODS | NATIX NETWORK - AUDIT

This report has been prepared for NATIX to discover issues and vulnerabilities in the source code of the NATIX Network - audit project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | NATIX NETWORK - AUDIT

System Overview

This audit concerns the implementation of NATIX Network with stake_v1 program.

External Dependencies

The project mainly contains the following dependencies:

Dependency	Version
solana-program	=2.0.9
borsh	1.3.1
spl-token	6.0.0
spl-associated-token-account	5.0.1

It should also be noted here that the code dependencies are in active development in the current auditing version and some of the keywords/functionality may be deprecated in a newer version. It is necessary to keep the dependencies up-to-date to avoid potential vulnerabilities.

The on-chain program can be upgradeable after the initial deployment due to Solana's features. Also, based on the unique rent mechanism in Solana, the balance in the account should be carefully set.

We assume these dependencies are valid and non-vulnerable factors and implement proper logic to collaborate with the current project.

Privileged Functions

The current project relies on an owner authority to ensure the dynamic runtime updates of the project, which are specified in the finding **GLOBAL-01**.

Any compromise to the privileged roles may allow the hacker to take advantage of this authority and disrupt the staking process.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plans to invoke a privileged function should also be considered to move to the execution queue of a `Timelock` contract.

The Solana platform allows for the possibility of upgrading its programs, with the default upgrade authority being the entity responsible for deployment. In situations where the program has upgradability features and the account of the upgrade authority becomes compromised, there is the potential for an unauthorized and malicious update to the program.

FINDINGS | NATIX NETWORK - AUDIT



14	0	1	3	3	7
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for NATIX Network - audit. Through this audit, we have uncovered 14 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
GLOBAL-01	**Centralization Related Risks And Upgradability**	**Centralization**	**Major**	**Acknowledged**
COS-01	Weak Validation On Token Account Owner	Volatile Code	Medium	Resolved
CVB-01	Compilation Failure	Coding Issue	Medium	Resolved
LIB-03	Potential Missing Link Between `program_account` And `stake_account`	Design Issue	Medium	Resolved
CHA-01	Unbounded Range For The Interest Rate	Design Issue	Minor	Resolved
SRC-03	Missing Owner Validation On `user_account` And `stake_account`	Volatile Code	Minor	Resolved
WIT-01	Potential Incorrect `forfeit_amount` Implementation	Design Issue	Minor	Resolved
CAN-01	Potential Missing Reward Handling While Gap Between `unstake` And `cancel`	Volatile Code	Informational	Acknowledged
GER-01	Unknown Source Of The Reward Tokens	Design Issue	Informational	Acknowledged
GER-02	Concerns On The Reward Calculation	Design Issue	Informational	Acknowledged
GIV-01	Concerns On Compute Unit And Account Size	Coding Issue	Informational	Acknowledged

ID	Title	Category	Severity	Status
LIB-01	Missing Address Check For Token Program	Logical Issue	Informational	Resolved
LIB-02	Missing Validation On Mint Account For `stake_token_account`	Volatile Code	Informational	Resolved
SRC-02	Cancellation May Be Blocked Due To Stake Amount Limit Validation And Break `maximum_stakers` Limitation	Volatile Code	Informational	Resolved

GLOBAL-01 CENTRALIZATION RELATED RISKS AND UPGRADABILITY

Category	Severity	Location	Status
Centralization	**Major**		**Acknowledged**

Description

In the program `stake_v1`, the role `owner` has authority over the following functions:

- `burn_or_transfer`: burn or transfer forfeit tokens
- `change_interest_rate`: update interest rate setting. **Note:** the reward rate would combine with the compound period due to the `get_reward` calculation, which should be carefully set by the team.
- `init`: init the program account with default setting
- `pause`: pause the program from staking, unstake, cancel, withdraw
- `resume`: resume from the program pause
- `set_config`: overwrite the config setting, which can affect the interest distribution and program functionalities. **Note:** since Since type cast(u64 to f64) may happen amount validation, the config value should be carefully set.

Any compromise to these accounts may allow a hacker to take advantage of this authority, manipulate the program functionalities, and cause asset loss.

Also, the Solana platform allows for the possibility of upgrading its programs, with the default upgrade authority being the entity responsible for deployment. In situations where the program has upgradability features and the account of the upgrade authority becomes compromised, there is the potential for an unauthorized and malicious update to the program.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[NATIX, 09/06/2024]: Unfortunately, Time-lock privileged operations is not feasible. As it's still an experimental program, we would like to be able to change the configs and access a few special operations.

However, we will move the program's owner and admin of the special operations to a Multisig account.

The following account will be used for Multisig in the future:

https://app.squads.so/squads/Ewbt8FTk39iAXuymJNdfvL6wEfU91ochNSbAdc5KWHkc/home

[CertiK, 09/06/2024]: CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

COS-01 | WEAK VALIDATION ON TOKEN ACCOUNT OWNER

Category	Severity	Location	Status
Volatile Code	Medium	src/control_user_solana_address.rs (08/22-67986 (Preliminary Report)): 23	Resolved

Description

The function `control_user_solana_address` in `src/control_user_solana_address.rs` performs a weak validation on the ownership of the user's associated token account. The current validation only checks if the user_account.key matches the expected address, but it doesn't ensure that the user_account is actually owned by the user.

```
25      if user_account.key != &address {
26          return Err(ProgramError::InvalidAccountOwner);
27      }
```

In Solana, associated token accounts can be transferred to other users, which means that the owner of the token account may not be the original user who created it. This weak validation can potentially allow unauthorized access or transfers of the associated token account.

Reference: https://github.com/solana-labs/solana-program-library/issues/2640

Recommendation

To strengthen the validation, it is recommended to also check if the `user_account` is owned by `user_solana_account` with `TokenAccount::owner` field.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue and added validation on token account owner in commit 3db9356d300f1850fc31ba0df844b1aecb094225.

CVB-01 | COMPILATION FAILURE

Category	Severity	Location	Status
Coding Issue	Medium	Cargo.toml (08/22-67986 (Preliminary Report)): 12	Resolved

Description

The project failed to compile, and some of the dependencies may have problems. The details are as following dependency：

```
spl-associated-token-account = { version = "2.3.1", features = [ "no-entrypoint" ]}
```

The issue should related to the version update on solana-program which caused the version of `spl-associated-token-account` to be yanked, more details can be found: https://github.com/solana-labs/solana-program-library/issues/6897

Recommendation

Recommend the team double checking Cargo.toml files to ensure all desired dependencies are correctly specified and the project compiles.

Alleviation

[NATIX, 09/10/2024]: The team acknowledged this issue and updated dependencies in commit dd18be8dcb430031e40de0a26a1de28f0ab9ddd6.

LIB-03 POTENTIAL MISSING LINK BETWEEN `program_account` AND `stake_account`

Category	Severity	Location	Status
Design Issue	Medium	src/lib.rs (08/22-67986 (Preliminary Report)): 85	Resolved

Description

In the current implementation, the `program_account` refers to the pool account that holds staking information, while the `stake_account` refers to the token account that holds the staked tokens and rewards.

However, the program allows multiple `program_account` and `stake_account` instances, which can cause confusion between different pools and token accounts.

For example, UserA could stake using `program_account1` and `stake_account1`, but then withdraw using `program_account1` and `stake_account2`.

To prevent this, each `program_account` should be explicitly linked to a specific `stake_account` to avoid any potential confusion.

Recommendation

We would like to check with the team if aforementioned scenario is considered.

Alleviation

[NATIX, 09/10/2024]: The team acknowledged this issue and updated the codebase in commit dd18be8dcb430031e40de0a26a1de28f0ab9ddd6 by:

1. using pda to build the connection between program_account and its stake_account.
2. adding `program_account` and its `stake_account` in config and validating while processing instructions.

CHA-01 | UNBOUNDED RANGE FOR THE INTEREST RATE

Category	Severity	Location	Status
Design Issue	Minor	src/change_interest_rate.rs (08/22-67986 (Preliminary Report)): 40	Resolved

Description

The current implementation of the staking program lacks validation or constraints while setting the staking rate. Consequently, it's possible to set a value greater than 100, allowing for an interest rate exceeding 100%.

Recommendation

We would like to check with team about the range of the interest to avoid unexpected reward distribution.

Alleviation

[NATIX, 09/18/2024]: The team acknowledged this issue and added validation on rate in commit 2bf22d4622ecf96a88cf3950d59e0cc706f5312f based on business design.

SRC-03 MISSING OWNER VALIDATION ON `user_account` AND `stake_account`

Category	Severity	Location	Status
Volatile Code	Minor	src/control_user_account.rs (08/22-67986 (Preliminary Report)): 19; src/lib.rs (08/22-67986 (Preliminary Report)): 85	Resolved

Description

The `user_account` and `stake_account` are treated as token accounts associated with the `natix_token_mint` .

But before decoding them as a `TokenAccount` , there is no validation to confirm that its owner is the **Token Program**, which allows the user to input a malicious token account.

Recommendation

Recommend adding owner validation on the `user_account` and `stake_account` with the `AccountInfo::owner` field to ensure it belongs to the Token Program.

Alleviation

[NATIX, 09/10/2024]: The team acknowledged this issue and adding owner validation on `user_account` / `stake_account` in commit dd18be8dcb430031e40de0a26a1de28f0ab9ddd6.

WIT-01 POTENTIAL INCORRECT `forfeit_amount` IMPLEMENTATION

Category	Severity	Location	Status
Design Issue	Minor	src/withdraw.rs (08/22-67986 (Preliminary Report)): 72	Resolved

Description

In the `withdraw.rs` file, line 72 calculates the total amount of tokens to be forfeited during a withdrawal in the staking contract:

```
72  forfeit_amount = forfeit_amount.checked_add((penalty.checked_sub(reward)).
unwrap_or(0)).expect("Overflow during forfeit summation");
```

However, there is a concern with this implementation:

The penalty amount is subtracted from the reward amount using `penalty.checked_sub(reward)`. This means that an amount of tokens equal to the reward will remain in the stake account, mixing with the user's deposit and project rewards since it is not specifically tracked.

Recommendation

We would like to check with the team if this is intended design.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue and updated `forfeit_amount` calculation in commit 3db9356d300f1850fc31ba0df844b1aecb094225.

CAN-01 POTENTIAL MISSING REWARD HANDLING WHILE GAP BETWEEN `unstake` AND `cancel`

Category	Severity	Location	Status
Volatile Code	Informational	src/cancel.rs (08/22-67986 (Preliminary Report)): 38~41	Acknowledged

Description

The current implementation of the unstake cancellation feature in `src/cancel.rs` does not update the `stake_time` when an unstake request is canceled. This oversight allows users to exploit the system by canceling their unstake request and claiming the full reward, even after initiating the unstake process and remaining in the unstake status.

```
38      for staker in stakes {
39          staker.unstake_time = 0;
40          staker.instant = false;
41      }
```

For example:

1. Alice stakes 1000 tokens at time T0 by calling the `stake` function.
2. At time T1, Alice initiates an unstake request by calling the `unstake` function, signaling her intention to withdraw her staked tokens and rewards. The staking program records Alice's `unstake_time` as T1.
3. At time T2, Alice cancels the unstake request using the `cancel` function.
4. Because `stake_time` is not updated, Alice's stake is still treated as if it began at T0. As a result, she receives rewards for the period between T1 and T2, while retaining flexibility during this time.

Recommendation

Recommend considering the gap period between `unstake` and `cancel` when calculating the reward.

Alleviation

[NATIX, 09/06/2024]: This is intentional and as per our requirements.

we would like to give an advantage to people who have changed their mind and cancel their unstacking process.

As their token has been locked anyway, it will not be unfair for others.

GER-01 | UNKNOWN SOURCE OF THE REWARD TOKENS

Category	Severity	Location	Status
Design Issue	Informational	src/get_reward.rs (08/22-67986 (Preliminary Report)): 9	Acknowledged

Description

In the `stake_V1` program, there is no specific logic or mechanism to track/deposit reward tokens. Insufficient rewards, potentially causing withdrawal blocks and losses in staking deposits.

Recommendation

We would like to check with team if any documents can be referenced about reward token deposit.

Alleviation

[NATIX, 09/06/2024]:

The method here is pre-calculated reward for each period. based on `max_staking_amount`, `max_stakers` we estimated the reward for different periods (e.g. 6 month) and pre-fill the stake account.

We are also building off-chain monitoring tools to monitor the health of the program and the balance of stake account and all wallet connect.

GER-02 | CONCERNS ON THE REWARD CALCULATION

Category	Severity	Location	Status
Design Issue	Informational	src/get_reward.rs (08/22-67986 (Preliminary Report)): 37	Acknowledged

Description

Based on current implementation, we have raised following concerns:

1. Potential for Excess Reward if No Rate Exists Before Stake Time

The current implementation may overestimate rewards when there's no interest rate defined before or at the stake time.

- If no rate is defined before the stake time, `get_interest_rate` only returns rates starting from or after the stake time.
- In `get_reward`, the first rate in this list is used as the initial rate.
- If this first rate is actually from a time after the stake time, it will be incorrectly applied to the entire period from stake time to this rate's time.
- This effectively gives the user a higher interest rate for a period when no rate was actually defined.

Example:

- Stake time: Day 0
- First defined rate: 5% starting from Day 10
- Unstake time: Day 30

In this case, the 5% rate would be applied for the entire 30-day period, including the first 10 days when no rate was actually defined, resulting in an overpayment of rewards.

2. Potential for Reward Loss Due to Compound Period The compound period can lead to underpayment of rewards due to rounding and missed rate changes.

- Different compound periods can result in different total rewards for the same staking duration and rates.
- Longer compound periods may cause more significant rounding losses:
    - Interest is only calculated at the end of each full compound period.
    - Any partial periods are essentially ignored, leading to lost interest.

3. Potential Precision Loss Due to Type Cast The reward calculation also includes type conversion between `u64` and `f64`. The conversion could lead to precision loss in certain scenarios especially when the value of u64 (i.e., `unstaker.amount`) exceeds the range of `f64`.

For example, in the following case, the value converted back to u64 increasing due to precision loss.

```
fn main() {
    let u64value: u64 = 123_456_789_123_456_780;
    let f64value: f64 = u64value as f64;
    let u64back: u64 = f64value as u64;
    println!("u64value: {:?}", u64value);
    println!("f64value: {:?}", f64value);
    println!("u64back: {:?}", u64back);
    println!("delta: {:?}", u64back - u64value);
}
```

Output

```
u64value: 123456789123456780
f64value: 1.2345678912345678e17
u64back: 123456789123456784
Panic with overflow
```

As a result, an underflow may occur. While the impact is limited to the actual staking amount, which is related to the staking limitation, it's recommended to use the `checked_*` method to handle extreme cases and limit it to 0 while overflow happens.

Proof of Concept

The following test is trying to explain the point 1:

simulation program:

```
use std::cmp::Ordering;
use std::f64::consts::E;
use std::ops::{Div, Mul};

#[derive(Clone, Debug, PartialEq)]
pub struct Rate {
    pub time: i64,
    pub amount: f64,
}

#[derive(Clone, Debug)]
pub struct StakeInfo {
    pub amount: u64,
    pub stake_time: i64,
    pub unstake_time: i64,
}

#[derive(Clone, Debug)]
pub struct Config {
    pub compound_period: u64,
}

pub fn get_interest_rate(rates: &[Rate], from_time: i64, to_time: i64) -> Vec<&Rate>
{
    let mut before_from_rates: Vec<&Rate> = rates.iter().filter(|rate| rate.time <
from_time).collect();
    before_from_rates.sort_by(|a, b| a.time.cmp(&b.time));

    let mut res: Vec<&Rate> = vec![];

    if before_from_rates.last().is_some() {
        res.push(before_from_rates.last().unwrap());
    }
    let rates: Vec<&Rate> = rates.iter().filter(|rate| from_time <= rate.time &&
rate.time <= to_time).collect();
    res.extend(rates);
    println!("rates: {:?}", res);
    res
}

pub fn get_reward(rates: &[Rate], stake_info: &StakeInfo, compound_period: u64) ->
u64 {
    let rates = get_interest_rate(rates, stake_info.stake_time,
stake_info.unstake_time);

    let mut t = stake_info.stake_time;
    let mut p = stake_info.amount as f64;
    let mut r: f64 = if rates.first().is_some() { rates.first().unwrap().amount }
else { 0.0 };
```

```
    for rate in rates {

        let diff_days: f64 = ((rate.time - t) as f64 / (60.0 * 60.0 * 24.0)) as f64;

        let clamped = (diff_days as u64).div(compound_period as
u64).mul(compound_period as u64);

        if clamped > 0 {
            p = p.mul(E.powf(r.mul((clamped as f64).div(365.0))));
            t = rate.time;
        }
        r = rate.amount;

    }

    let diff_days: f64 = ((stake_info.unstake_time - t) as f64 / (60.0 * 60.0 *
24.0)) as f64;

    let clamped = (diff_days as u64).div(compound_period as u64).mul(compound_period
as u64);

    if clamped > 0 {
        p = p.mul(E.powf(r.mul((clamped as f64).div(365.0))));
    }

    ((p as u64) - stake_info.amount).max(0)
}
```

test:

```
    #[test]
    fn test_get_reward_simple() {
        let rates = vec![
            Rate { time: 120 * 24 * 60 * 60, amount: 0.05 }, //@audit stake in the
middle still get the full
        ];

        let stake_info = StakeInfo {
            amount: 1000,
            stake_time: 60 * 24 * 60 * 60,
            unstake_time: 365 * 24 * 60 * 60, // 1 year
        };

        let reward = get_reward(&rates, &stake_info, 1);
        println!("reward: {:?}", reward); // 51
    }
```

Recommendation

We would like to check with the team if the aforementioned scenario has been considered.

Alleviation

[NATIX, 09/06/2024]:

- The scenario of having no rate for the first stake is not valid. When we deploy the contract we set the config with interest rates. without an interest rate, there is not logical to have a staking program setup.
- The second scenario is valid. We might lose some interest due to the round mathematics function, however, our compound period will be less than 10 days, the loss should be only on decimals (our token will support 6 digit decimals)

[NATIX, 09/11/2024]: The team updated the codebase in commit 9bf1e6e4f3b190b28e077baba901858c7aa9565b with checked method to avoid unexpected type cast result.

[CertiK, 09/18/2024]: Based on the update, u64 to f64 cast is being used in multiple places, including stake amount validation(`control_stake_amount`), the team should carefully set up the config values as well to avoid unexpected cast.

GIV-01 | CONCERNS ON COMPUTE UNIT AND ACCOUNT SIZE

Category	Severity	Location	Status
Coding Issue	Informational	src/give_stakes.rs (08/22-67986 (Preliminary Report)): 28	Acknowledged

Description

Solana maintains limitations on both account size and compute units:

- Account size: 10 MB, reference
- Compute unit: 1_400_000(200_000), reference

The concern is that the current implementation uses a vector to hold all stake information. Although the number of stakes is constrained with `maximum_stakers`, however, it limits the unique user, which means it allows one staker/user to create numbers of stakes which could potentially abuse the account size.

At the same time, the program frequently go through the whole stake list, which raise the concern when too many stake info exists.

Recommendation

We would like to check with the team if scenario above is considered.

Alleviation

[NATIX, 09/06/2024]: Our aim was to have the storage of information decentralised. we have looked at few option and Solana account was still most. We are aware of the limitation and we handle the size by introducing max_staking_amount, max_stakers and etc.

[CertiK, 09/06/2024]: Ideally, the program should avoid storing all staking information in a single account. For instance, it would be better to allow each owner to manage their own staking account. However, implementing this would likely require significant refactoring of the current design.

It would be advisable to limit the number of staking items as well. Currently, the project allows a single staker to insert multiple stake items while only restricting the number of unique stakers, rather than the total number of stake items.

[NATIX, 09/10/2024]: For the current version, we have discussed again to control unique stakers, but we don't see the need.

This is a limited staking program with limited reward. It's based on a first come, first serve approach, and in less than 6 month we will have new version.

It's really doesn't matter for us whether those wallets' stakes are unique or not. as we have a minimum stake amount and in total, only few thousand wallets that own $NATIX, we believe we can allow multiple staking for each wallet.

[CertiK, 09/10/2024]: As mentioned by the team the number of the staking can be limited by the `min_stake_amount` (per stake), `max_stake_amount` (per stake and per user) and `maximum_stakers` (unique staker). To prevent unexpected staking behavior, the team should carefully set these limits.

LIB-01 | MISSING ADDRESS CHECK FOR TOKEN PROGRAM

Category	Severity	Location	Status
Logical Issue	Informational	src/lib.rs (08/22-67986 (Preliminary Report)): 78	Resolved

Description

The address check is intended to verify that the input token program account is the legitimate token program (i.e., `spl_token::id()`) on Solana. However, the provided `token_program_account` does not perform this check, raising concerns about the validity of the account input by the user.

Recommendation

Recommend adding validation on the input token program account.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue and added validation on the given `token_program_account` in commit 3db9356d300f1850fc31ba0df844b1aecb094225.

LIB-02 MISSING VALIDATION ON MINT ACCOUNT FOR `stake_token_account`

Category	Severity	Location	Status
Volatile Code	Informational	src/lib.rs (08/22-67986 (Preliminary Report)): 85	Resolved

Description

The mint validation for `stake_token_account` is missing which may allow user input an non-NATIX token account.

Recommendation

Recommend adding mint address validation for `stake_token_account`.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue and added validation on `stake_account` (pool token account) in commit 3db9356d300f1850fc31ba0df844b1aecb094225.

SRC-02 CANCELLATION MAY BE BLOCKED DUE TO STAKE AMOUNT LIMIT VALIDATION AND BREAK `maximum_stakers` LIMITATION

Category	Severity	Location	Status
Volatile Code	Informational	src/cancel.rs (08/22-67986 (Preliminary Report)): 43; src/control_stake_amount.rs (08/22-67986 (Preliminary Report)): 38; src/give_stakes.rs (08/22-67986 (Preliminary Report)): 33	Resolved

Description

The `cancel` function in cancel.rs allows users to cancel all their unstake requests. However, the function includes a call to `control_stake_amount` with None as the amount parameter after giving back the unstakes to the user's stake pool. The `control_stake_amount` function validates if the total stake amount for the user's address is within the configured limit.

```
38      if sum_stakes > staking_pool.config.max_stake_amount {
39          return Err(ProgramError::Custom(102));
40      }
```

If canceling the unstake requests causes the total stake to exceed the limit, the control_stake_amount function would return an error, blocking the cancellation process. This means that users may be unable to cancel their unstake requests if doing so would cause their total stake to exceed the allowed limit.

At the same time, the limitation on unique stakes (`config.maximum_stakers`) can be broken due to cancellation when a new user joins between the `unstack` and `cancel` periods.

```
32      for staker in staking_pool.stakers.iter().filter(|staker|
33          staker.unstake_time == 0 && staker.stake_time > 0) {
34              set.insert(staker.address);
35          }
36
37      set.len()
```

Recommendation

We would like to check with the team if the aforementioned **two** scenario has been considered.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue and confirmed both scenarios should stop the process of cancelation based on design, and added quantity validation on unique staker in commit

3db9356d300f1850fc31ba0df844b1aecb094225.

[CertiK, 09/06/2024]: The user should aware the potential failure of cancellation during the usage.

OPTIMIZATIONS | NATIX NETWORK - AUDIT

ID	Title	Category	Severity	Status
SRC-01	Potential Conflict On Setting	Logical Issue	Optimization	Acknowledged

SRC-01 | POTENTIAL CONFLICT ON SETTING

Category	Severity	Location	Status
Logical Issue	Optimization	src/burn_or_transfer.rs (08/22-67986 (Preliminary Report)): 57, 67; src/set_config.rs (08/22-67986 (Preliminary Report)): 21	Acknowledged

Description

The `burn_or_transfer` function in `src/burn_or_transfer.rs` checks the `pool.config.forfeit_allow_burn` and `pool.config.forfeit_allow_transfer` flags to determine whether to burn or transfer the forfeited tokens.

However, there is no validation during the `set_config` operation to ensure that only one of these flags can be set to true at a time.

As a result,

1. If both `pool.config.forfeit_allow_burn` and `pool.config.forfeit_allow_transfer` are true, the code will prioritize burning the forfeited tokens over transferring them.
2. If both `pool.config.forfeit_allow_burn` and `pool.config.forfeit_allow_transfer` are true, the forfeited tokens will be locked.

Recommendation

Recommend adding validation to ensure one of the `pool.config.forfeit_allow_burn` and `pool.config.forfeit_allow_transfer` is true.

Alleviation

[NATIX, 09/06/2024]: The team acknowledged this issue, and this setup was intentional and based on our requirements.

APPENDIX | NATIX NETWORK - AUDIT

Finding Categories

Categories	Description
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

